

บทที่ 6

อาร์เรย์หลายมิติ

(Multi-Dimension Array)



คณาจารย์ภาควิชาวิศวกรรมคอมพิวเตอร์
introcom@coe.psu.ac.th

อาร์เรย์หลายมิติ

อาร์เรย์หลายมิติ (Multi-dimensional array) คือ อาร์เรย์ที่มีสมาชิกเป็นข้อมูลอาร์เรย์ นั่นคือ ในหน่วยข้อมูลแต่ละหน่วยของอาร์เรย์ จะเป็นอาร์เรย์ย่อยๆ ซึ่งอาจจะกำหนดซ้อนลงไปได้หลายชั้น

การกำหนดอาร์เรย์หลายมิติ จะกระทำในรูป

ชนิดตัวแปร ชื่อตัวแปร[จำนวนสมาชิก][จำนวนสมาชิก]....;

ตัวอย่างอาร์เรย์หลายมิติ

เช่น อาร์เรย์ที่มีสมาชิกอยู่ 3 ตัว และสมาชิกแต่ละตัวก็เป็นอาร์เรย์เก็บข้อมูลชนิด int มีจำนวนสมาชิก 2 ตัว จะกำหนดได้ดังนี้

```
int a[3][2];
```

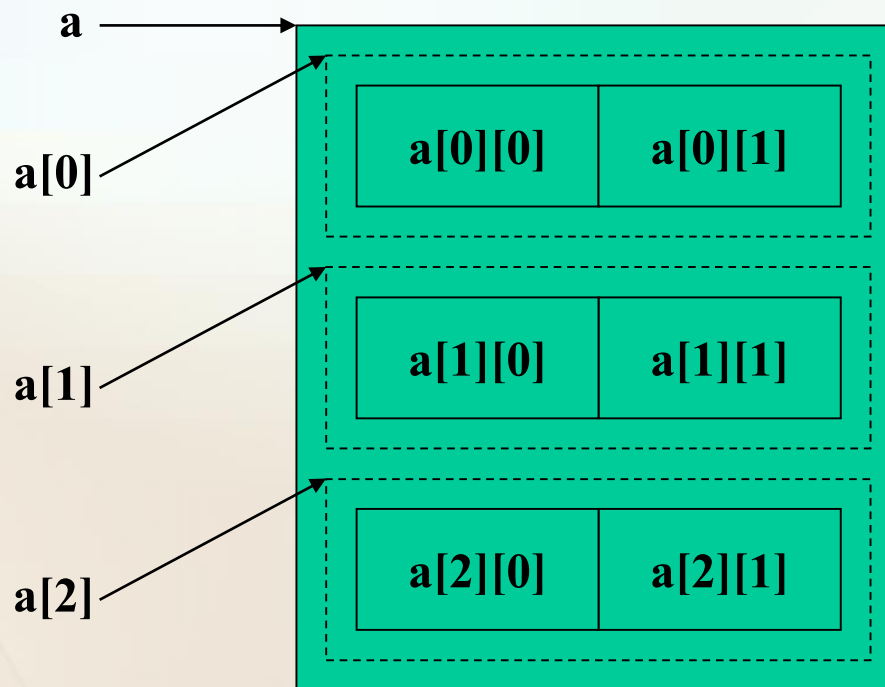
- ขนาดของอาร์เรย์ 3 x 2
- จำนวนไบต์ ที่ใช้ในการเก็บอาร์เรย์

`sizeof (a)` คำนวณจาก $3 \times 2 \times \text{sizeof}(\text{int}) = 3 \times 2 \times 4 = 24$ ไบต์

ตัวอย่างอาร์เรย์หลายมิติ

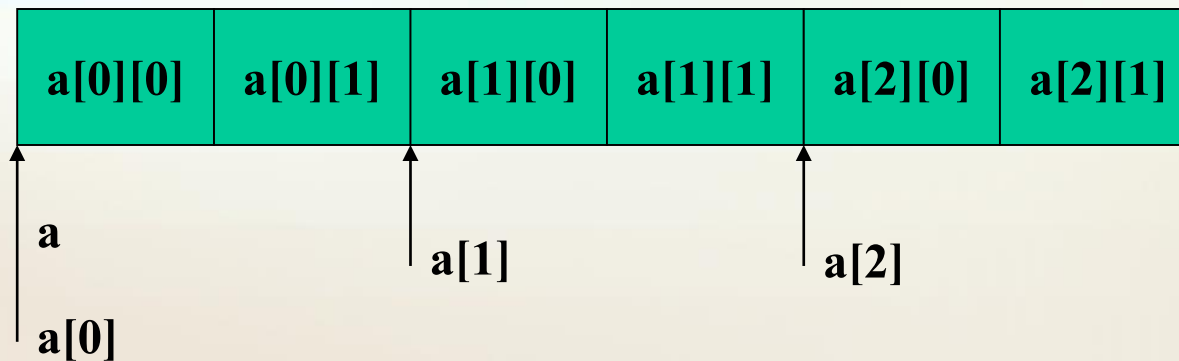
จากการกำหนดดังกล่าว จะได้อาร์เรย์ที่มีโครงสร้างดังรูป

```
int a[3][2];
```



ตัวอย่างอาร์เรย์หลายมิติ

การจัดวางภายในหน่วยความจำ สามารถแสดงได้ดังรูป



อาร์เรย์แบบ 2 มิติ

อาร์เรย์แบบ 2 มิติ มีโครงสร้างคล้ายคลึงกับเมตริกซ์สองมิติ จะเป็นตัวแปรที่มีการอ้างอิงถึงค่าข้อมูลโดยใช้ค่าเลขดัชนี 2 ค่า ซึ่งประกอบไปด้วยค่าดัชนีที่ใช้ในการอ้างอิงในแนวแถว (rows) และค่าดัชนีที่ใช้อ้างอิงในแนวคอลัมน์ (columns) ตัวอย่างข้อมูลอาร์เรย์ 2 มิติ เช่น

8	16	9	52
3	15	27	6
14	25	2	10

```
int val[3][4];      //(4 bytes * 3 * 4 = 48 bytes)
```

ตัวอย่างที่ 11 : การดูขนาดอาร์เรย์ 2 มิติในหน่วยไบต์

- ไฟล์ arrayex11.c

```
#include <iostream>
int main()
{
    int x[2][5];
    char names[3][4];
    float nums[2][2];
    cout<<"Size of x = " << sizeof(x)<<endl;
    cout<<"Size of names ="<<sizeof(names)<<endl;
    cout<<"Size of nums = "<<sizeof(nums)<<endl;
    return 0;
}
```

Size of x = 40
Size of names = 12
Size of nums = 16

การเข้าถึงอาร์เรย์แบบ 2 มิติ

```
int a[3][2];
```

- สมาชิกของอาร์เรย์ `a` แต่ละตัว คือ `a[0]` `a[1]` และ `a[2]` ประกอบด้วยสมาชิกที่เป็นตัวแปรเดี่ยวชนิด `int` อยู่สองตัว
- การเข้าถึงสมาชิกแต่ละตัว จะต้องเขียนชื่ออาร์เรย์แล้วตามด้วยลำดับข้อมูลที่ล้อมด้วยเครื่องหมาย `[ ]`
- ดังนั้น การเข้าถึงสมาชิกที่มีค่าหมายเลขลำดับเป็น 1 ของอาร์เรย์ `a[0]` จึงเขียนอยู่ในรูป `a[0][1]`

อาร์เรย์แบบ 2 มิติ

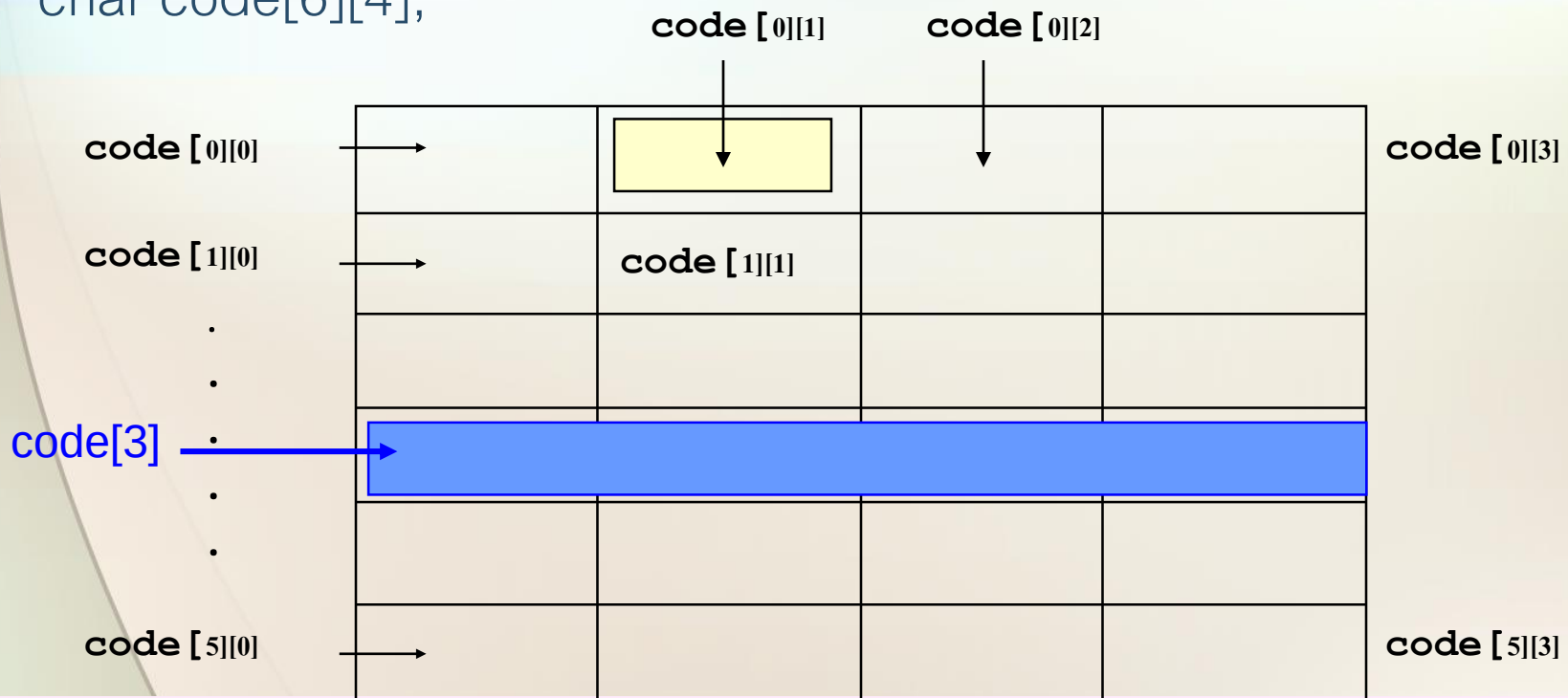
- โดยสรุป สำหรับอาร์เรย์สองมิติ เมื่ออ้างชื่ออาร์เรย์ จะหมายถึงตำแหน่งเริ่มต้นของอาร์เรย์ทั้งหมด (อาร์เรย์ 2 มิติ)
- เมื่ออ้างชื่ออาร์เรย์พร้อมสมาชิกหนึ่งอันดับ จะหมายถึงตำแหน่งเริ่มต้นของอาร์เรย์ย่อยภายใน (อาร์เรย์ 1 มิติ)
- เมื่ออ้างชื่ออาร์เรย์พร้อมค่าสองอันดับ จะหมายถึง ข้อมูลภายในอาร์เรย์

การประกาศตัวแปรอาร์เรย์ 2 มิติ

```
int val[3][4];
```

```
double prices[10][5];
```

```
char code[6][4];
```



ตัวอย่างการเข้าถึงตัวแปรอาร์เรย์ 2 มิติ

```
num = val[3][4];
```

```
val[0][0] = 8;
```

```
new_nu = 4 * ( val[1][1] - 5);
```

```
sum_row0 = val[0][0] + val[0][1] + val[0][2] + val[0][3];
```

	Col.0	Col.1	Col.2	Col.3	
Row 0	8	16	9	52	
Row 1	3	15	27	6	← val[1][3]
Row 2	14	25	2	10	

Row position Column position

การใช้คำสั่ง *for* ในการเข้าถึงอาร์เรย์ 2 มิติ

- ใช้ลูป **for 2 ชั้น** โดยลูปชั้นนอกวนรอบตามจำนวนแถว ส่วนลูปชั้นในวนรอบตามจำนวนหลัก
- ต้องมีตัวนับ 2 ตัว คือ ตัวนับแถวและตัวนับหลัก
- ตัวอย่างเช่น

```
int i,j,x[2][3];
for(i=0;i<2;i++)
    for(j=0;j<3;j++)
        x[i][j] = i+j;
```

	0	1	2
0	0	1	2
1	1	2	3

ตัวอย่างที่ 12 : การแสดงค่าของอาร์เรย์ 2 มิติ

- ไฟล์ arrayex12.c

```
#include<iostream>
int main(void)
{
    int val[3][4]={8,16,9,52,3,15,27,6,14,25,2,10};

    cout<<"Display all elements of array"<<endl;

    cout<<val[0][0]<<" "<<val[0][1]<<" "<<val[0][2]<<" "<<val[0][3]<<endl;
    cout<<val[1][0]<<" "<<val[1][1]<<" "<<val[1][2]<<" "<<val[1][3]<<endl;
    cout<<val[2][0]<<" "<<val[2][1]<<" "<<val[2][2]<<" "<<val[2][3]<<endl;

    for (int i =0; i < 3; ++i){
        cout<<endl; /* start a new line for each row */
        for (int j =0; j < 4; ++j)
            cout<<val[i][j]<<" ";
    }
    return 0;
}
```

Display all elements of array

```
8 16  9 52
3 15 27  6
14 25  2 10
```

```
8 16  9 52
3 15 27  6
14 25  2 10
```

ตัวอย่างที่ 13 : การแสดงค่าของอาร์เรย์ 2 มิติ

- ไฟล์ arrayex13.c

```
#include <iostream>
int main()
{
    int val[3][4]={8,16,9,52,3,15,27,6,14,25,2,10};
    /*multiply each element by 10 and display it */

    cout<<"Display of multiplied elements"<<endl;

    for(int i=0 ; i < 3 ; ++i ){
        cout<<endl; /*start a new line for each row */
        for(j =0; j < 4; ++j){
            val[i][j]=val[i][j]*10;
            cout<<val[i][j]<<"  ";
        } /*End for j */
    } /*End for i */

    return 0;
} /*End main */
```

Display all multiplied elements

80	160	90	520
30	150	270	60
140	250	20	100

ตัวอย่างที่ 15 : การรับและแสดงค่าของอาร์เรย์ 2 มิติ

- ไฟล์ arrayex15.c

```
#include <iostream>
int main() {
    int a[2][3];
    for(int i=0;i<2;i++){
        for(int j=0;j<3;j++){
            cout<<"Enter a["<<i<<" ] ["<<j<<" ] ";
            cin>>a[i][j];
        }
    }
    for(int i=0;i<2;i++){
        for(int j=0;j<3;j++){
            cout<<a[i][j]<<"  ";
        }
        cout<<endl;
    }
    return 0;
}
```

```
Enter a[0][0]: 10
Enter a[0][1]: 20
Enter a[0][2]: 30
Enter a[1][0]: 40
Enter a[1][1]: 50
Enter a[1][2]: 60
10 20 30
40 50 60
```

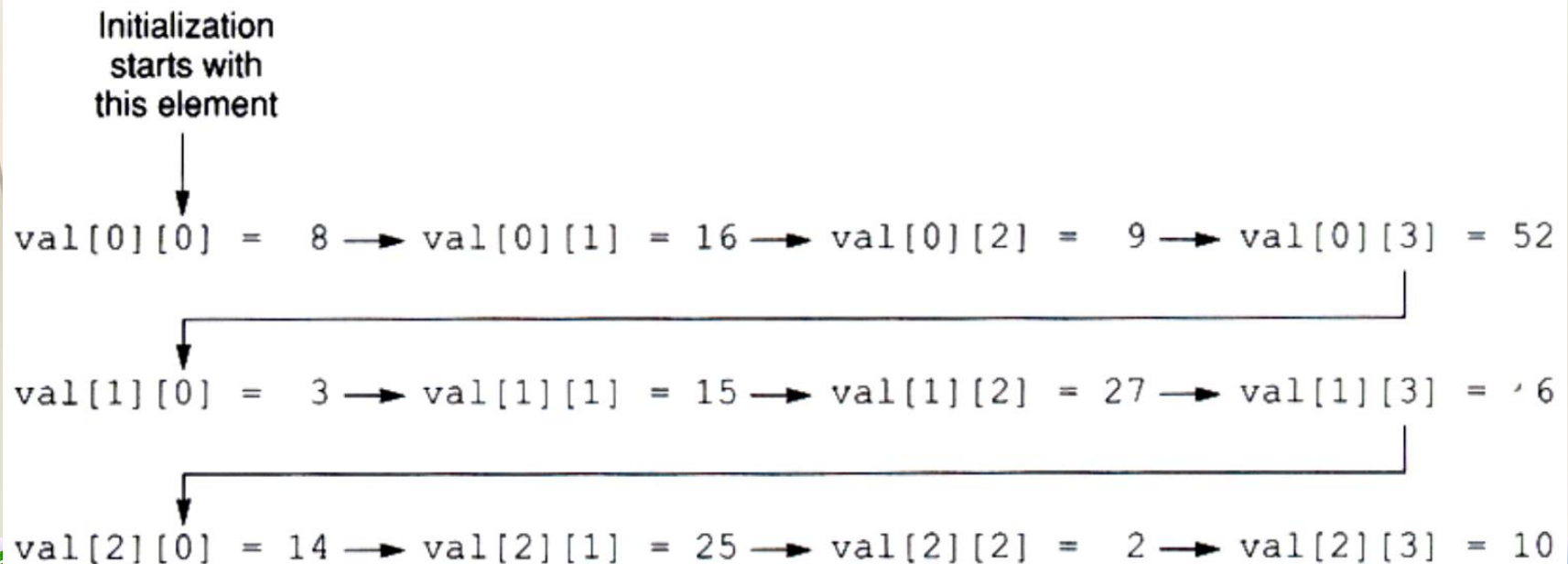
การให้ค่าเริ่มต้น (Array Initialization)

- เราจะใช้กลุ่มค่าคงที่ที่มีสมาชิกเป็นกลุ่มค่าคงที่ย่อย ซึ่งเป็นชนิดเดียวกันและมีขนาดเท่ากัน รวมถึงสอดคล้องกับชนิดของอาร์เรย์ด้วย
- โดยใช้เครื่องหมาย {} หรือ , ในการแบ่งแยกแต่ละแถว

ตัวอย่างการให้ค่าเริ่มต้น (Array Initialization)

```
int val[3][4]  = { { 8,16, 9, 52 },
                  {3,15, 27, 6},
                  {14, 25, 2, 10} };

int val[3][4]  = { 8, 16, 9, 52,
                  3, 15, 27, 6,
                  14, 25, 2, 10  };
```



- หากไม่มีการกำหนดจำนวนแถว คอมไพเลอร์จะกำหนดจำนวนแถวโดยนับจากที่กำหนดในค่าเริ่มต้น แต่จะต้องมีการกำหนดจำนวนหลักเสมอ เช่น

```
int a[][2] = {{5,8},{9},{-1}};
```

	0	1
0	5	8
1	9	0
2	-1	0

- ในการกำหนดค่าเริ่มต้นของอาร์เรย์ 2 มิติ สามารถละเครื่องหมายปีกกาที่ใช้แบ่งแถวได้ โดยใช้จำนวนหลักในการจัดว่าอีลีเมนต์ใดอยู่แถวใด

เช่น `float x[2][3] = { 2.5, 1.25, 8.6, 4.1, 6.9, 7.2 };`

	0	1	2
0	2.5	1.25	8.6
1	4.1	6.9	7.2

โจทย์ฝึกสมองที่ 3 : ไฟล์ arrayprac3.c

- จงเขียนโปรแกรมสำหรับรับ **เมตริกซ์** ของเลขจำนวนเต็มขนาด 2x2 จำนวน 1 เมตริกซ์ จากนั้นแสดงค่า Determenant ของ array A

$$\det(A) = (a_{11}a_{22} - a_{12}a_{21})$$

- ตัวอย่างผลการรันเป็นดังนี้ (ข้อความสีแดงคือค่าที่รับจากผู้ใช้)

Enter matrix A(2x2) :

1 2

3 4

The determenant of A is -2

```

#include <iostream>
Using namespace std;
int main()
{
    int A[2][2];
    cout<<"Enter matrix A :";
    for(int i=0;i<2;i++){
        // for(j=0;j<2;j++) cin>>A[i][j]; รับทีละ element
        cin>> A[i][0], A[i][1];
    }
    cout<<"Determenent of A is";

    .....
    .....

    return 0;
} //end of main

```



ฟังก์ชันที่รับค่าเข้าเป็นอาร์เรย์ 2 มิติ

- จากต้นแบบของฟังก์ชัน:

```
void display(int val[3][4]);
```

- หมายถึง ฟังก์ชันชื่อ display ไม่มีการส่งค่ากลับ แต่รับค่าเข้าเป็นอาร์เรย์ของ int ขนาด 3 แถว 4 หลัก
- หรืออาจจะละจำนวนแถวก็ได้ แต่ต้องระบุจำนวนหลักเสมอ เช่น

```
void display(int val[ ][4]);
```

ตัวอย่างที่ 16 : ฟังก์ชันที่รับค่าเข้าเป็นอาร์เรย์ 2 มิติ

- ไฟล์ arrayex16.c

```
#include <iostream>
void display(int nums[3][4]);    /* function prototype */
int main()
{
    int val[3][4] = {8,16,9,52,3,15,27,6,14,25,2,10};
    display(val);
    return 0;
}

void display (int nums[3][4])
{
    for(int row_num = 0 ; row_num < 3 ; row_num++){
        for(int col_num = 0; col_num < 4; ++col_num)
            cout<<nums[row_num][col_num]<<" ";
        cout<<endl;
    } /* End for col_num */
}
```

8	16	9	52
3	15	27	6
14	25	2	10

อาร์เรย์ที่มากกว่า 2 มิติ

เช่น อาร์เรย์ 3 มิติ

```
int volume[3][4][2];
```

```
int i,j,k;
```

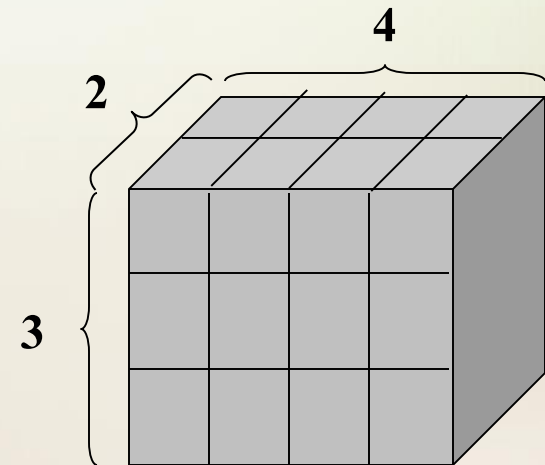
```
for(i=0;i<3;i++)
```

```
for(j=0;j<4;j++)
```

```
for(k=0;k<2;k++)
```

```
val[i][j][k] = 0;
```

- อีลีเมนต์แรกคือ `val[0][0][0]`
- อีลีเมนต์สุดท้าย `val[2][3][1]`



ความผิดพลาดของโปรแกรมทั่วไปเกี่ยวกับอาร์เรย์

- ลืมประกาศตัวแปร
- ใช้งานตัวแปรอาร์เรย์ นอกเหนือจากขอบเขตที่ประกาศไว้

```
int a_error[3] , i;
```

```
for ( i = 0 ; i <=3 ; i++ )
```

```
    a_error[i] = i;
```

ความผิดพลาดของโปรแกรมทั่วไปเกี่ยวกับอาร์เรย์ (ต่อ)

- ลืมให้ค่าเริ่มต้นกับตัวแปรอาร์เรย์ แล้วนำตัวแปรนั้นไปใช้งาน

```
int a1 = 5, a2 = 8;
```

```
int total[2];
```

```
total[1] = total[0] + a1 + a2;
```

การให้ค่าเริ่มต้นอย่างง่าย *int array[10][5][3]={ };* ทุก element เป็นศูนย์

- กำหนดค่าให้กับตัวแปรอาร์เรย์โดยตรง

```
int x[10];
```

```
x = 11;
```

การเรียงข้อมูลใน Array 2D

```
#include <iostream>
using namespace std;
void selectionSort2D(int arr[][4], int rows, int cols)
{
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols - 1; j++) {
            int minIndex = j;
            for (int k = j + 1; k < cols; k++) {
                if (arr[i][k] < arr[i][minIndex]) {
                    minIndex = k;
                }
            }
            // Swap the elements
            if (minIndex != j) {
                int temp = arr[i][j];
                arr[i][j] = arr[i][minIndex];
                arr[i][minIndex] = temp;
            }
        }
    }
}
```

Before Sorting 2D array:

5 2 9 7

8 1 6 2

3 7 4 1

Sorted 2D array:

2 5 7 9

1 2 6 8

1 3 4 7

```
int main() {
    int matrix[][4] = {{5, 2, 9, 7}, {8, 1, 6, 2}, {3, 7, 4, 1}};
    int rows = 3;
    int cols = 4;
    cout << "Before Sorting 2D array:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    // Call the selectionSort2D
    selectionSort2D(matrix, rows, cols);
    cout << "Sorted 2D array:" << std::endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```



แบบฝึกหัด (ไฟล์ arrayprob2.c)

- จงเขียนโปรแกรมสำหรับรับเมตริกซ์จากผู้ใช้ 1 เมตริกซ์ (ให้ชื่อว่าเมตริกซ์ A) โดยผู้ใช้งานสามารถกำหนดจำนวนหลักและแถวของเมตริกซ์ได้ (แต่สมมติว่าผู้ใช้ใส่ค่าจำนวนแถวและจำนวนหลักไม่เกิน 10) จากนั้นให้แสดงผลลัพธ์ของเมตริกซ์ $2A$
- ตัวอย่างผลการรันโปรแกรมเป็นดังนี้ (ข้อความสีแดงคือค่าที่รับจากผู้ใช้)

```
Enter number of rows: 2
Enter number of columns: 3
Enter matrix A:
4 3 5
2 9 7
Matrix 2A:
8 9 10
4 18 14
```



Homework เมตริกซ์ทรานสโพส Transpose Matrix

- เมตริกซ์ทรานสโพสของเมตริกซ์ A คือ เมตริกซ์ที่ได้จากการสลับตำแหน่งแถวและหลัก ถ้า A มีขนาด $m \times n$ เมตริกซ์ทรานสโพสของเมตริกซ์ A จะมีขนาด $n \times m$

ให้ B คือ transpose matrix ของ A

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix}$$

$$b_{11} = a_{11}, b_{12} = a_{21}, b_{13} = a_{31}$$

$$b_{21} = a_{12}, b_{22} = a_{22}, b_{23} = a_{32}$$

- จงเขียนโปรแกรมสำหรับรับเมตริกซ์จากผู้ใช้ 1 เมตริกซ์ (ให้ชื่อว่าเมตริกซ์ A) โดยผู้ใช้สามารถกำหนดจำนวนหลักและแถวของเมตริกซ์ได้ (แต่สมมติว่าผู้ใช้ใส่ค่าจำนวนแถวและจำนวนหลักไม่เกิน 10) จากนั้นให้แสดงผลลัพธ์เป็นเมตริกซ์ทรานสโพสของ A

Homework โปรแกรมรับข้อมูลสินค้า เพื่อดำเนิน VAT และราคาสุทธิ

ข้อกำหนด : โดยจัดเก็บในตัวแปร **ARRAY** 1 มิติ คือ **name[5]** และ 2 มิติ ประกอบด้วย **price[5][3]**

main() รับค่าสินค้าและราคาทั้งหมด

เรียกใช้ **calprice(price)**

เรียกใช้ **display(name,price)**

function calprice(float price[5][3]) ทำหน้าที่คำนวณภาษี และราคารวมสุทธิ เก็บค่าไว้ใน **array** ของ **price**
display(string name[5],float price[5][3]) แสดงผลตามตัวอย่าง

```
(Inactive C:\C412\ARRAY2D2.EXE)
Enter Product Name : mouse
      price : 100
Enter Product Name : keyboard
      price : 120
Enter Product Name : CDRom
      price : 1000
Enter Product Name : LCD
      price : 3500
Enter Product Name : Speaker
      price : 500
```

No.	product	price	vat7%	Net
1	mouse	100.00	7.00	107.00
2	keyboard	120.00	8.40	128.40
3	CDRom	1000.00	70.00	1070.00
4	LCD	3500.00	245.00	3745.00
5	Speaker	500.00	35.00	535.00

